Comparing Monte Carlo and Sarsa(λ) on Flappy Bird

Henri Boisson
Student Number: B00819225

ESSEC Business School / CentraleSupélec
Reinforcement Learning, March 2026

Abstract. This report compares two reinforcement learning agents on Text Flappy Bird: a Monte Carlo (MC) agent and a Sarsa(λ) agent with eligibility traces, both trained on `TextFlappyBird-v0`. MC achieves higher peak scores (mean $\approx$21, max 199) but is unstable; Sarsa(λ) is more consistent (mean $\approx$11, max 105) and generalises much better to new configurations.

1 Experimental Setup

1.1 Environment

I used `TextFlappyBird-v0` (height=15, width=20, pipe_gap=4). The observation is $[d_x, d_y]$: horizontal and vertical distance to the next pipe gap. The agent flaps (1) or does nothing (0), receiving $+1$ per pipe cleared and -1 for crashing.

1.2 State Representations

MC — Q-table. I discretised $[d_x, d_y]$ into a 20×20 grid, storing one Q-value per (state, action) pair: it is a simple lookup table updated as the agent plays.

Sarsa(λ) — Linear approximation. Instead of a table, I used features derived from $[d_x, d_y]$: normalised distances, squared vertical distance, its sign, their product, and a bias. The agent learns a weight per feature; the action-value is their dot product.

1.3 Algorithms

Monte Carlo. MC plays a full episode then updates each visited Q-value using the actual total reward received from that point:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[G_t - Q(S_t, A_t)\big]. \tag{1}$$

Sarsa(λ). Sarsa(λ) updates at every step using a TD error and eligibility traces to credit earlier states:

$$\mathbf{z}_t = \gamma\lambda\,\mathbf{z}_{t-1} + \boldsymbol{\phi}(S_t, A_t), \qquad \mathbf{w} \leftarrow \mathbf{w} + \alpha\,\delta_t\,\mathbf{z}_t. \tag{2}$$

λ controls how far back credit is assigned: $\lambda = 0$ is one-step TD, $\lambda = 1$ approaches MC.

Hyperparameters. Both agents: $\gamma = 0.99$, ε decreased from $1.0 \to 0.01$ over 3 000 episodes. MC: $\alpha = 0.1$. Sarsa(λ): $\alpha = 0.05$, $\lambda = 0.7$ (from sweeps, Section 2.3).

2 Results

2.1 Learning Curves

Both agents improve over the first 1 500 episodes (Fig. 1). MC reaches higher scores (max 199, rolling mean up to 34) but oscillates heavily. Sarsa(λ) stabilises around a mean score of 11. MC's variability comes from using the full episode return, which is accurate but noisy; Sarsa(λ) uses shorter-horizon estimates that are more stable at the cost of some bias.

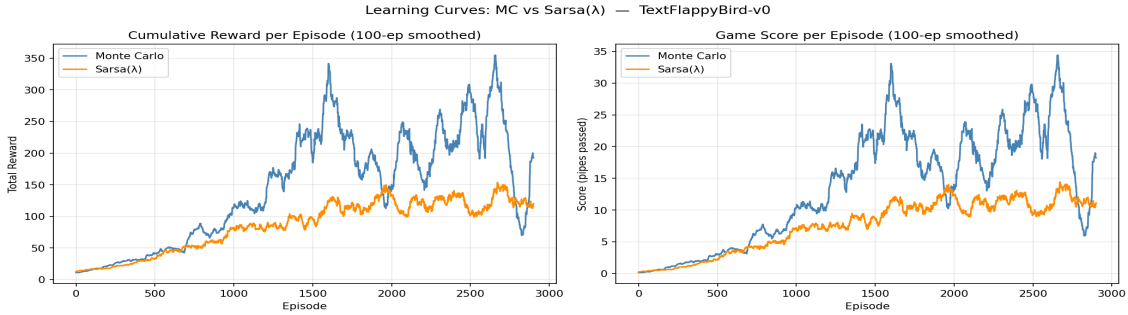
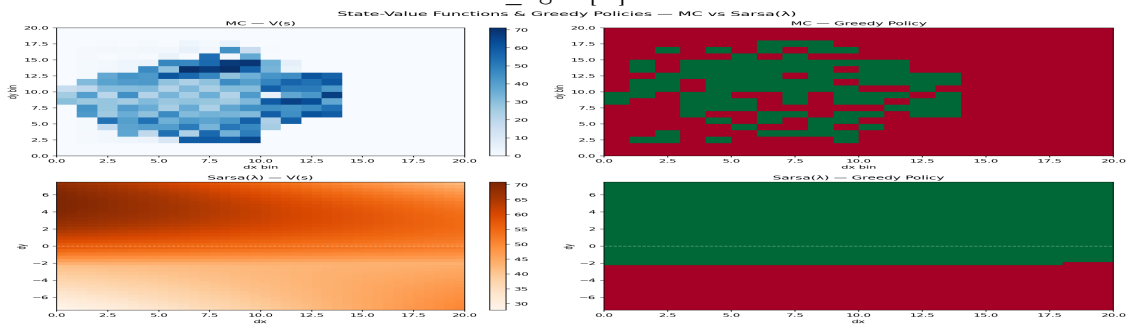


Fig. 1: 100-episode rolling mean reward (left) and score (right). MC = blue, Sarsa(λ) = orange.

2.2 State-Value Functions and Policies

_figure[h!]



The MC heatmap (Fig. 2.2) only has non-zero values where the agent visited: the rest is blank. The policy is noisy because the Q-table has no generalisation between nearby bins. Sarsa(λ) produces a smooth value function and a clean policy: flap above the gap ($d_y > 0$), do nothing below ($d_y < 0$): the correct control law emerging from the linear weights.

2.3 Parameter Sweeps

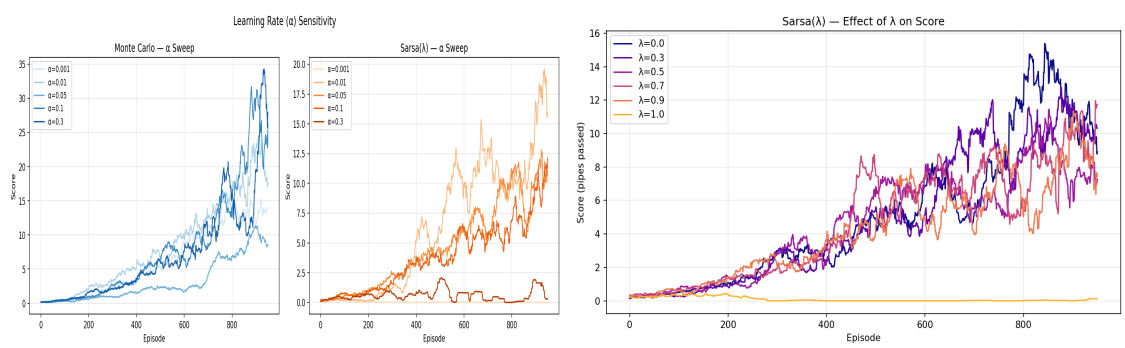


Fig. 2: Left: α sweep (both agents, 1 000 eps). Right: λ sweep for Sarsa(λ).

MC is robust across $\alpha \in [0.001, 0.3]$; best at $\alpha = 0.1$ (Fig. 2). Sarsa(λ) diverges at $\alpha = 0.3$. The λ sweep shows that $\lambda = 1.0$ fails entirely: traces never decay and the weights blow up. $\lambda = 0.0$ performs best; intermediate values are comparable, so $\lambda = 0.7$ was kept as a safe default.

3 Discussion

3.1 Environment Versions

I chose `TextFlappyBird-v0` because its two-number observation fits both a Q-table and a linear model. The screen version `TextFlappyBird-screen-v0` gives the full text grid as input, the state

space is enormous and both agents scored near zero in preliminary experiments. A neural network would be needed there.

3.2 Applicability to the Original Flappy Bird Environment

`flappy-bird-gym` uses pixel images. The same MC and Sarsa(λ) ideas still apply, but a Q-table or linear model cannot handle raw pixels. A CNN is required, this is how DQN works on the TD side, and actor-critic methods on the MC side.

3.3 Generalisation Across Configurations

Table 1: Mean $\pm$ std score over 200 greedy episodes. Trained on gap=4, h=15, w=20.

Configuration	MC mean	MC std	Sarsa mean	Sarsa std
Trained (gap=4)	4.20	3.22	0.84	1.16
Easier (gap=6)	4.41	4.02	499.00	0.00
Harder (gap=2)	0.72	0.98	0.00	0.00
Taller (h=20)	2.87	3.02	1.07	1.71
Wider (w=30)	0.00	0.00	1.17	1.41

Sarsa(λ) scores **499** on gap=6 (Table 1), essentially solving it by hitting the step limit. Its linear model generalises smoothly to unseen states. MC is stuck to visited bins and scores only around 4. Both agents struggle when the grid size changes, as $[d_x, d_y]$ values shift outside the trained range.

3.4 Summary

Table 2: Summary comparison.

Property	Monte Carlo	Sarsa(λ)
When it updates	End of episode	Every step
Bootstrapping	No	Yes
State representation	Q-table	Linear weights
Mean score (last 500)	21.1	11.1
Max score	199	105
Best α	0.1	0.05
Best λ	N/A	0.0–0.3
Generalisation	Poor	Strong (gap=6: 499)

MC reaches higher peaks but is noisy and needs a full episode before learning anything. Sarsa(λ) is steadier, updates continuously, and generalises far better though it needs careful tuning of λ to avoid divergence (Table 2).

Full code: `https://github.com/henridollars/RL-AssignmentIndiv-FlappyBird`